

1. **Elements and attributes:** What HTML is for · Elements, tags and comments · Attributes · Nesting and the document tree
2. **The document:** The document skeleton · What goes in the head
3. **Headings, text and lists:** Headings and the outline · Paragraphs and text-level elements · Lists
4. **Semantic HTML:** Why semantic HTML · The landmark elements
5. **Block, inline, div, span:** Block and inline elements · div and span: generic containers
6. **Links and images:** Links and paths · Images and alt text
7. **Tables:** Tables for tabular data
8. **Forms:** Forms: names, buttons and submitting · Form controls: labels, choices and text areas · Built-in form validation
9. **Validate and inspect:** Validate and inspect

Elements and attributes

What HTML is for

HTML (HyperText Markup Language) describes the structure and meaning of a page: it marks each piece of content as a heading, a paragraph, a list, a link and so on.

- **Markup:** plain text with **tags** in angle brackets around the content (see [Elements, tags and comments](#)).
- **Hypertext:** text with links to other documents. The `<a>` element is what turns separate pages into a web.
- **Not a programming language:** no variables, conditions or loops. Looks come from [CSS](#) and behaviour from [JavaScript](#) (see [How it all fits together](#)).
- **What the browser does with it:** downloads the file, builds a tree of elements from it and shows it with built-in default styles (see [how the browser renders](#)).

Example. The words "Opening hours" can be a heading (`<h2>Opening hours</h2>`), a paragraph or a list item. Only as a heading does a screen reader announce "heading level 2, Opening hours" and let the user jump to it with one key. Same words, different meaning, and the markup is what carries it.

Common mistake. Choosing tags by how they look: `<h3>` because "the text should be smaller", `<blockquote>` to indent a paragraph. The look is CSS's job and can be changed in one line; the tag must say what the content **is**.

Elements, tags and comments

An element is a start tag, its content and an end tag (`<p>Hello</p>`); the tags are only the markers in angle brackets.

Void elements No content and no end tag: `<br>`, `<img>`, `<input>`, `<meta>`, `<link>`, `<hr>`.
Older code writes `<br />`; in HTML that final slash is allowed but ignored.

Comments `<!-- ... -->` are notes for people. The browser does not display them, but anyone can read them with View source, so never put passwords or private notes there.

Letter case Tag names are not case-sensitive (`<P>` works like `<p>`), but always write lowercase.

Example. In `<a href="squad.html">See the squad</a>`: the **start tag** is `<a href="squad.html">` (it carries an attribute), the **content** is "See the squad", the **end tag** is `</a>`, and all three together are the **element**.

Common mistake. Relying on the browser to close your tags. Browsers repair missing end tags silently, so the page often looks right, but the repaired tree may not be what you meant (text ending up inside the wrong element), the validator reports errors, and CSS and JavaScript later target the wrong elements.

Attributes

Attributes add information or settings to an element; they are written as `name="value"` pairs inside the start tag, separated by spaces.

Attributes you will use on your first pages

Attribute	On	Purpose
href	<a>, <link>	Where the link points (see Links and paths)
src	, <script>	Which file to load
alt		Text alternative for the image
width, height		Display size in pixels; reserves the space
lang	Any element	Language of the content
title	Any element	Advisory tooltip text

id or class

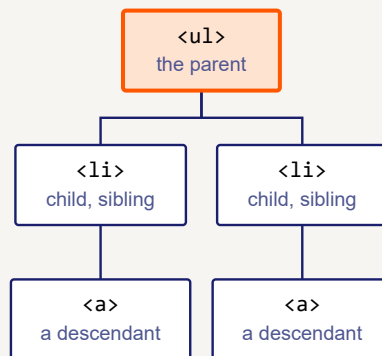
- **id is unique:** exactly one element per page has it. Fragment links (`href="#intro"`), form labels (see [Form controls](#)) and JavaScript all expect one match.
- **class is a group:** any number of elements share it, and one element can have several, separated by spaces (`class="lead highlight"`). CSS selects groups by it (see [CSS selectors](#)).

Example. `` has four attributes: the file to load, the text alternative, and the size the browser reserves for it. Their order does not matter.

Common mistake. Giving the same `id` to several elements (`id="card"` on every card). An `id` must be unique: a fragment link, a form label or JavaScript expects exactly one match. For a group of similar elements use a class: `class="card"`.

Nesting and the document tree

Elements nest inside each other and form a **tree**; each element must be closed inside the element that contains it, never overlapping.



Every element has exactly one parent: the nesting is a tree.

Parent The element that directly contains another: the `<ul>` is the parent of each `<li>`.

Child An element directly inside another.

Siblings Children of the same parent: the two `<li>` elements.

Descendant Anything inside, at any depth: each `<a>` is a descendant of the `<ul>`.

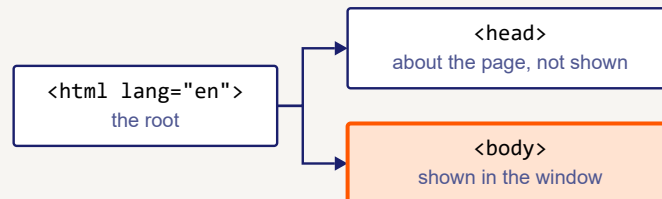
Example. In `<p>Read the <a href="news.html"><strong>latest</strong> news</a>.</p>`, the `<p>` is the parent of the `<a>`, the `<a>` is the parent of the `<strong>`, and the `<strong>` is a descendant of the `<p>` two levels down.

Common mistake. Putting a list or a heading inside a `<p>`. The browser ends the paragraph as soon as it meets the `<ul>`, so the list is not inside the paragraph, and the leftover `</p>` creates an extra empty paragraph. The Try-it box shows three bordered paragraphs where the code seems to have two.

The document

The document skeleton

Every HTML page starts with `<!DOCTYPE html>` and has one `<html>` element containing a `<head>` (information about the page) and a `<body>` (the content shown in the window).



One root, two children: information about the page, and the page itself.

- `<!DOCTYPE html>` A declaration, not an element: "this is modern HTML, use **standards mode**". Without it, browsers switch to **quirks mode**, imitating the bugs of 1990s browsers, and some CSS behaves differently. It comes first in the file; only comments and blank lines may precede it.
- `<html lang="en">` The language of the content. Screen readers pick their pronunciation from it, browsers offer translation, and search engines show the page to the right audience. Use `lang="es"` for a page in Spanish.

HTML

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Home · Riverside FC</title>
  </head>
  <body>
    <h1>Riverside FC</h1>
    <p>Welcome to the official club website.</p>
  </body>
</html>

```

Example. Open this skeleton in a browser: the tab shows "Home · Riverside FC" (from `<title>`, in the head), and the window shows only the heading and the paragraph (from the body). The indentation is for people; the browser ignores it.

Common mistake. Leaving out `<!DOCTYPE html>` because "the page looks fine without it". The difference shows up later: in quirks mode some CSS sizing rules behave like in very old browsers, and the validator reports an error. Start every file with it.

What goes in the head

The `<head>` holds **metadata**, data about the page: the character encoding, the viewport settings, the title, a description, and links to stylesheets and scripts; none of it is displayed as page content.

`<meta charset="utf-8">`

The file is encoded in **UTF-8**, which covers every alphabet, accents and emoji. Without it, "España" can appear as "Españ±a".

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

On phones, lay the page out at the real screen width instead of a shrunken desktop page (see [responsive foundations](#)).

```
<title>
```

The text of the browser tab, of bookmarks and of the link in search results; screen readers read it first. **Required:** the validator reports an error without it.

```
<meta name="description" content="...">
```

A one-sentence summary that search engines may show under the title.

```
<link rel="stylesheet" href="styles.css">
```

Connects an external CSS file (see [Writing CSS](#)).

```
<script src="app.js" defer></script>
```

Loads a JavaScript file and runs it after the HTML has been read (see [Loading scripts](#)).

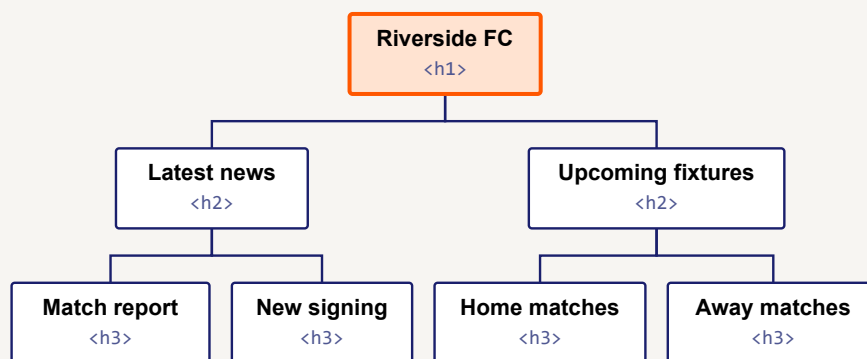
Example. The fixtures page of a club site: `<title>Fixtures · Riverside FC</title>` in the head names the tab, while `<h1>Fixtures and results</h1>` in the body is the visible heading. A good title puts the specific part first, so it still reads well when many tabs are open and truncated.

Common mistake. Mixing up three different "titles": the `<title>` element (in the head, names the document in the tab), the `<h1>` element (in the body, the visible main heading) and the `title` attribute (a tooltip on one element). They have different places and different jobs.

Headings, text and lists

Headings and the outline

`<h1>` to `<h6>` mark headings of six levels; together they form the **outline** (the table of contents) of the page, so the level must follow the structure, not the font size you want.



The headings are the page's table of contents.

- **One `<h1>` per page:** the page title. The standard allows more, but one is the common, safest convention, and the one this site follows.
- **Never skip a level going down:** after an `<h2>` comes an `<h3>`, not an `<h4>`. Going back up (an `<h4>`, then a new `<h2>`) is fine.
- **Size is only a default style:** CSS can change it, so never choose a level because of its size.

Example. The outline in the Try-it box reads like a table of contents: page title, two chapters, two sections in each. If "New signing announced" were an `<h4>`, the outline would have a level-4 heading directly under a level-2 one, and a screen-reader user would wonder what was missing.

Common mistake. Picking `<h4>` for a subtitle because "it has the right size", or using a bold paragraph as a fake heading. The first breaks the outline; the second is invisible to heading navigation. Choose the level by structure and adjust the size with CSS.

Paragraphs and text-level elements

Running text goes in **paragraphs** (`<p>`); inside a line, text-level elements add meaning: `<strong>` (importance), `<em>` (stress emphasis), `<a>` (link), `<br>` (a line break that belongs to the content).

<code></code>	"This is important": a warning, a key fact.
<code></code>	"Stress this word", which can change the meaning of a sentence.
<code></code> , <code><i></code>	Set text apart without extra importance (a product name, a word in another language). Rarely what you need.
<code>
</code>	A line break that is part of the content , as in a postal address or a poem; never a spacing tool.
<code><hr></code> , <code><blockquote></code>	A change of topic between paragraphs; a quotation from another source.

Example. "I did **not** say that" and "I did not say **that**" mean different things when spoken; `<em>` marks where the stress goes: `<p>I did not say that.``</p>`. A warning that must not be missed is ``.

Common mistake. Using `<br><br><br>` or empty `<p></p>` to create vertical space. They add meaningless empty lines to the content, and the spacing cannot adapt to the screen. Separate the content into proper elements and add space with CSS margin (see [the box model](#)).

Lists

`<ul>` makes an **unordered (bulleted) list** and `<ol>` an **ordered (numbered) list**; each item is an `<li>`. A nested list goes inside the `<li>` it belongs to.

- **Only `<li>` children:** `<ul>` and `<ol>` accept nothing else directly; a sub-list goes inside an `<li>`.
- **Counting:** `<ol start="4">` begins at 4, and reversed counts down.
- **Menus are lists of links:** CSS later removes the bullets and lines the items up, but the meaning ("a list of 5 links") stays.
- **Terms and descriptions** (a glossary, the details of a match) use a description list: `<dl>` with `<dt>` (term) and `<dd>` (description).

Example. A recipe page: the ingredients are a `<ul>` (the order you buy them in does not matter), the method is an `<ol>` (step 3 cannot come before step 1), and the site menu at the top is a `<ul>` of links inside a `<nav>` (see [The landmark elements](#)).

Common mistake. Placing a sub-list next to the items instead of inside one: `<ul><li>Goalkeepers</li><ul>...</ul></ul>`. A list may only contain `<li>` children, so the validator reports an error; move the inner `<ul>` inside the `<li>`, before its `</li>`.

Semantic HTML

Why semantic HTML

Semantic HTML means choosing elements whose name says what the content is (`<nav>`, `<article>`, `<button>`) instead of generic boxes (`<div>`, `<span>`); it makes pages accessible, understandable to search engines and easier to maintain.

- **Accessibility:** screen readers list the regions ("navigation", "main") and the headings, so users jump straight to what they need. A real `<button>` works with the keyboard for free; a clickable `<div>` does not (see [ARIA and keyboard focus](#)).
- **Search engines** read the title, the headings and the main content to understand what the page is about. Clear structure helps them; it is not a magic ranking boost.
- **Maintainability:** `</article>` tells a teammate what is closing; `</div></div></div>` does not.

HTML

```
<!-- "div soup": works, but means nothing -->
<div class="top">
  <div class="title">Riverside FC</div>
  <div class="menu">...</div>
</div>

<!-- semantic: the same page, with meaning -->
<header>
  <h1>Riverside FC</h1>
  <nav>...</nav>
</header>
```

Example. A screen-reader user opens both versions of this code. On the semantic one they hear "banner landmark, heading level 1, Riverside FC, navigation landmark" and can press a key to jump to the navigation. On the `<div>` version they hear only "Riverside FC" followed by the menu text, with no way to know it is a title or a menu.

Common mistake. Using semantic elements as decoration: wrapping every block in `<section>` or every image in `<article>`. Each element is a claim about the content, and a wrong claim misleads as much as none. When no element fits, a `<div>` is the honest choice.

The landmark elements

Landmark elements divide a page into regions: `<header>` (introduction, logo, site navigation), `<nav>` (major navigation links), `<main>` (the unique main content), `<section>` (a themed group with a heading), `<article>` (a self-contained piece), `<aside>` (related but secondary content) and `<footer>` (closing information).

Which landmark?

Element	Question to ask	Example
<code><header></code>	Is it the introduction of the page, or of an article?	Logo, site name, main menu
<code><nav></code>	Is it a major set of navigation links?	Main menu, table of contents
<code><main></code>	Is it the content this page exists for?	Everything between header and footer
<code><section></code>	Is it a themed group I can give a heading to?	"Upcoming fixtures"
<code><article></code>	Would it make sense on its own elsewhere?	A match report, a product card, a comment
<code><aside></code>	Is it related, but removable without loss?	League table widget, related links
<code><footer></code>	Is it closing information?	Copyright, contact, legal links

- **One** `<main>` per page, never inside `<header>`, `<nav>`, `<article>`, `<aside>` or `<footer>`.
- **A** `<section>` **has a heading**: if you cannot name it, it is probably a `<div>`.
- **The article test**: would it still make sense on its own, copied to another site or a news feed?
- `<header>` **and** `<footer>` **also work inside** an `<article>` or `<section>`: a post's title and date, its author line.
- `<nav>` **is for major navigation**, not every group of links.

HTML

```
<body>
  <header>
    <h1>Riverside FC</h1>
    <nav><!-- main menu --></nav>
  </header>
  <main>
    <section>
      <h2>Latest news</h2>
      <article>
        <h3>Riverside 2-1 Northside</h3>
        <p>A late goal sealed the win...</p>
      </article>
    </section>
    <aside>
```

```

    <h2>League table</h2>
  </aside>
</main>
<footer>© 2026 Riverside FC</footer>
</body>

```

Example. In this layout the match report is an `<article>` (it makes sense on its own, shared on social media), "Latest news" is a `<section>` (a group of reports with a heading), and the league table is an `<aside>` (useful, but the page still works without it). The semantic outline tool on this card draws the outline of any page you paste.

Common mistake. Choosing between `<section>` and `<article>` by size or position. The question is independence: a match report stands alone, so it is an `<article>`; "Upcoming fixtures" is a themed group with a heading, so it is a `<section>`. An article can contain sections and a section can contain articles.

Block, inline, div, span

Block and inline elements

By default, **block elements** (`<p>`, `<h1>`, `<ul>`, `<div>`, the landmarks) **start on a new line and take the full available width**; **inline elements** (`<a>`, `<strong>`, `<em>`, `<span>`, `<img>`) **flow inside a line of text and take only the width of their content**.

What may go inside what

- **Inline goes inside blocks:** `<p>Read the <a href="news.html">latest news</a>.</p>` is right; a `<p>` inside a `<span>` is invalid.
- **A `<p>` holds only text-level (inline) content:** never a list, a heading or a `<div>`.
- **Lists:** `<ul>` and `<ol>` take only `<li>` children; an `<li>` can contain almost anything, another list included.
- **The exception, `<a>`:** it may wrap block content (a whole card), but never another link or a button.

Example. In the Try-it box, the solid navy outlines (blocks) stretch across the whole width even when the text is short; the dashed orange outlines (inline) hug their words. Add `display: block;` to the `span` rule and watch the two spans move onto separate lines.

Common mistake. Putting a block element inside an inline one, e.g. wrapping a whole card `<div>` in a `<span>`, or an `<h2>` in an `<em>`. Inline elements may contain only text-level content; the browser has to repair the tree and the validator reports an error.

div and span: generic containers

`<div>` (block) and `<span>` (inline) are **generic containers with no meaning**: use them to group content for styling or scripting only when no semantic element fits.

- `<div>`** A wrapper that exists only for layout: `<div class="card-grid">` so CSS can place cards side by side.
- `<span>`** A hook for part of a sentence that has no special meaning: a price to colour, `<span class="price">25 €</span>`, or a phrase in another language, `<span lang="es">¡Vamos!</span>`.

Example. In the Try-it box each ticket is an `<article>` (it stands on its own), but the wrapper that places them side by side is a `<div>`: it exists only for layout and has no meaning. The prices are `<span>`s because a price is not more important than the text around it; the class is just a styling hook.

Common mistake. Writing text in a bare `<div>` instead of a `<p>`. Once styled it looks the same, but it is no longer a paragraph for screen readers, reader modes and translation tools. Text in paragraphs goes in `<p>`; a `<div>` is for wrappers (whole pages built from `<div>`s are in [Why semantic HTML](#)).

Links and images

Links and paths

The `<a>` element makes a link; its `href` is an **absolute URL** (another site), a **relative path** (a file of your own site, found from the current page) or a **fragment** `#id` (a place on a page).

Kinds of `href`, seen from `index.html` in the site folder

href	Kind	Goes to
<code>https://www.w3.org/</code>	Absolute URL	Another site; the scheme <code>https://</code> is required
<code>squad.html</code>	Relative, same folder	The sibling file <code>squad.html</code>
<code>pages/tickets.html</code>	Relative, subfolder	Into the <code>pages</code> folder first
<code>../index.html</code>	Relative, parent folder	<code>..</code> means one folder up
<code>/index.html</code>	Root-relative	From the root of the server , not of your folder
<code>#results</code>	Fragment	The element with <code>id="results"</code> on this page
<code>fixtures.html#results</code>	File + fragment	Opens <code>fixtures.html</code> , scrolls to <code>id="results"</code>
<code>mailto:info@example.com</code>	Another scheme	Opens the email program

- **Link text says where it goes** ("See all fixtures"), never "click here": screen-reader users often pull up a list of all the links on a page, out of context.
- **File names are case-sensitive** on most web servers, including GitHub Pages (a free host for static sites): `Squad.html` and `squad.html` are different files there, although Windows treats them as one. Use lowercase names without spaces.
- **A fragment needs a target:** some element with a matching `id`.
- `target="_blank"` opens the link in a new tab; use it sparingly, for example for external documents.

Example. A site with `index.html`, `fixtures.html` and a folder `pages/` containing `tickets.html`. From `index.html`: `href="pages/tickets.html"`. From `pages/tickets.html` back home: `href="../index.html"`. From `index.html` to the results section of the fixtures page: `href="fixtures.html#results"`.

Common mistake. Using root-relative paths (`/squad.html`) when opening the files straight from your disk. With `file://`, `/` means the root of the **drive** (e.g. `C:/`), not your project folder, so every link breaks; they only work once the site is served. While you work from local files, use plain relative paths such as `squad.html`.

Images and alt text

`<img>` embeds an image: `src` is the path of the file, `alt` a text alternative, and `width` and `height` the display size in pixels so the browser can reserve its space before the file arrives.

- `alt="..."`** The information the picture gives. For a chart, that may be its conclusion.
- `alt=""`** A purely decorative image: screen readers skip it. Leaving the attribute out is different, and wrong: some screen readers then read the file name aloud.
- `width, height`** Plain numbers of pixels (`width="320"`, no `px`). They reserve the box before the file arrives, so the text below does not jump; CSS can still resize the image.
- `<figure>`, `<figcaption>`** An image with a visible caption.

Example. For the club crest, `alt="Riverside FC crest: a blue shield with a white river"` gives the information. If the crest is also the link to the homepage, describe the destination instead: `alt="Riverside FC home"`. A decorative wave graphic between sections gets `alt=""`.

Common mistake. Writing `alt="image"`, `alt="photo of..."` or the file name. Screen readers already say "image"; the `alt` must carry the information the picture gives. For a chart, that may be its conclusion: "Season ticket sales doubled between 2023 and 2025".

Tables

Tables for tabular data

A **table** shows data organised in rows and columns: `<table>` contains rows (`<tr>`) of header cells (`<th>`) and data cells (`<td>`), with an optional `<caption>` and the groups `<thead>` and `<tbody>`.

`<th scope="col">`, `scope="row"` A header cell for its column or its row, so a screen reader says "Opponent: Northside United" instead of reading a bare cell.

`<caption>` The title of the table.

`<thead>`, `<tbody>` Group the header rows and the data rows.

`colspan="2"`, `rowspan="2"` A cell that spans several columns or rows.

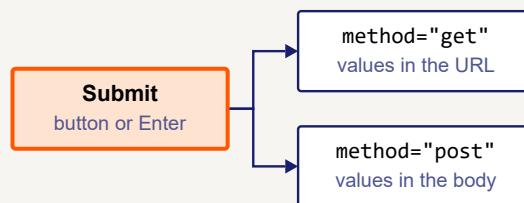
Example. A fixtures list has a date, an opponent and a venue for each match: a table. A squad list with only names is a list (`<ul>`), not a one-column table. A "logo on the left, menu on the right" header is layout, which is CSS.

Common mistake. Making header cells with bold `<td>`s, or using a table to put things side by side. Bold data cells look like headers but are not headers for assistive technology; side-by-side placement is layout, done with CSS [Flexbox](#) and [Grid](#).

Forms

Forms: names, buttons and submitting

A `<form>` groups controls whose values are sent together: each control's name is the key of its value, a submit button sends them, and `action` and `method` say where and how.



The same values travel in the URL with GET and in the request body with POST.

action The URL the request goes to. By default, **the current page**; on a real site, a server route that checks the data.

method get (the default) puts the values in the query string of the URL: fine for a search or a filter. post puts them in the request body: for log-ins and anything private or that changes data.

`<button type="submit">` Sends the form, and so does Enter in a text field. A `<button>` inside a form is a submit button by default, so always write `type="submit"` or `type="button"`.

Example. If this sign-in form used the default get method, submitting it would load `login.html?email=ana%40example.com&password=s3cret%21`: the password visible in the address bar and saved in the history. That is why a login form uses `method="post"`.

Common mistake. Believing a form without `action` "does nothing" when submitted. It **reloads the current page**, and with the default get method every named field, the password included, is appended to the URL. Before there is a server, give login forms `method="post"` and expect the page to reload.

Form controls: labels, choices and text areas

Every **form control** (a text box, a set of radio buttons, a drop-down list, a text area) needs a `<label>`, and its kind decides how the user answers and what is sent.

Which control?

You need	Use	Sent as
Short text	<code><input type="text"></code> , Or email, password, number, date	name=value
One choice of a few	Radio buttons sharing one name	The value of the checked one
Yes or no	<code><input type="checkbox"></code>	name=on when ticked; nothing when not
One choice of many	<code><select></code> with <code><option value="..."></code>	The chosen option's value
Long text	<code><textarea></code>	name= the text

- **The input** type changes the checks and the phone keyboard: email shows an @ key, password hides the characters, number brings a numeric keypad.
- **Radio groups:** the shared name makes them one group, so choosing one unchecks the others. Wrap the group in `<fieldset>` with a `<legend>`, the question for the whole group.

Example. Pick "Junior" in the Try-it box: "Adult" unchecks itself, because both radios share `name="ticket"`. Submitted, this form would send `ticket=junior&match=12-oct¬es=`, plus `newsletter=on` only if the box is ticked. The last line is written with `&` and `&t` in the source.

Common mistake. Giving each radio button its own name (`name="adult"`, `name="junior"`). They no longer form a group: the user can tick both, and the server receives two unrelated keys instead of one answer. Radios that answer one question share one name and differ in value.

Built-in form validation

HTML can check values before a form is sent: `required`, `minlength/maxlength`, `min/max`, `pattern` and **typed inputs** such as `type="email"` **make the browser block the submission and show a message.**

<code>required</code>	The field cannot be empty (a boolean attribute).
<code>minlength="8", maxlength="64"</code>	Limits on the number of characters of text.
<code>min="1", max="10"</code>	Limits on numbers and dates.
<code>pattern="[0-9]{5}"</code>	The value must match this regular expression , a text pattern: here, exactly five digits.
<code>type="email", url, number</code>	The value must have that format.

Example. In the Try-it box the empty required fields start red. Type abc as the password: it stays red because of `minlength="8"`; at the eighth character it turns green. The postcode is green while empty (it is not required) and red as soon as you type a letter.

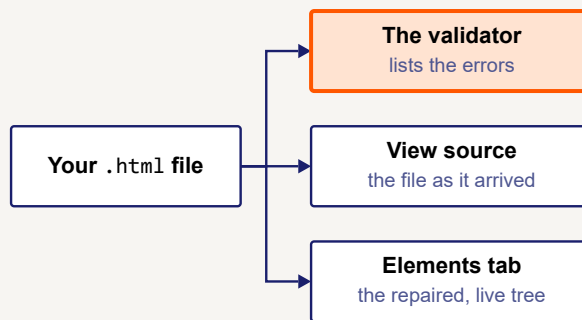
Common mistake. Treating `required` and `type="email"` as protection. They improve the experience, but a user can remove them in DevTools in seconds and a tool like `curl` ignores them completely. Real validation always happens on the server.

Validate and inspect

Validate and inspect

The **W3C Markup Validator** checks your HTML against the rules of the standard and lists its errors; the

browser **DevTools** (Elements tab) show the tree the browser actually built from it.



Three views of one page: the rules, the file, and what the browser built.

The validator

At validator.w3.org, choose **Validate by Direct Input** (or upload the file) and paste the whole document. **Errors** break the rules (overlapping tags, an `<li>` outside a list, a missing `<title>`, a duplicate `id`, an `<img>` without `alt`) and must be fixed. **Warnings** are advice worth reading. Fix the **first** error first: one missing end tag can cause a cascade of follow-up errors.

The Elements tab

Right-click an element and choose **Inspect**, or press F12. It shows the live tree (see [What the DOM is](#)); hovering a node highlights it on the page, and double-clicking edits it until the next reload.

View source

Ctrl+U (Cmd+Opt+U on macOS) shows the file exactly as it arrived.

Example. A page has `` with no `alt`. The image displays, and the Elements tab shows it as a normal `<img>`: nothing looks wrong. Only the validator reports "An img element must have an alt attribute", which a screen-reader user would have noticed at once.

Common mistake. Taking the Elements tab as a copy of your file. It shows the **repaired, live** DOM: missing end tags appear "fixed", and a `<tbody>` appears in every table even if you never wrote one. To see what you really wrote, use View source or your editor, and run the validator.